

MODUL PRAKTIK/PRAKTIKUM

IF451 – Advance Web Programming

PROGRAM SARJANA S1

FAKULTAS TEKNIK DAN INFORMATIKA

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung B Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia

Telp: +62-21.5422.0808, web: <http://www.umn.ac.id>



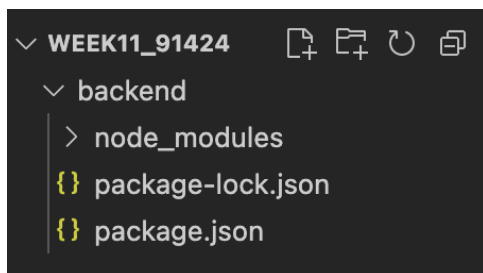
Modul Week 11

Connecting Express API to ReactJS

Persiapan Praktikum

1. Buatlah folder dengan nama **Week11_NIMtanpa0** dan didalamnya, buatlah folder **backend**. Jalankan command berikut pada terminal :

```
cd backend
npm init -y
npm install express nodemon
```



2. Buatlah folder **src** dan buat file **server.js** di dalam folder **src**. Tambahkan kode berikut pada **src/server.js**

```
backend > src > JS server.js > ...
1  const express = require('express');
2
3  const app = express();
4  const PORT = 5001;
5
6  // Middleware to parse JSON bodies
7  app.use(express.json());
8
9  // Sample route
10 app.get('/api/hello', (req, res) => {
11   res.status(200).json({ message: `Hello from ${req.method} request on ${req.originalUrl}` });
12 });
13
14 app.post('/api/hello', (req, res) => {
15   const { name } = req.body;
16   res.status(201).json({ message: `Hello, ${name}! This is a ${req.method} request on ${req.originalUrl}` });
17 });
18
19 app.put('/api/hello', (req, res) => {
20   const { name } = req.body;
21   console.log("Editing something....");
22   res.status(200).json({ message: `Hello, ${name}! This is a ${req.method} request on ${req.originalUrl}` });
23 });
24
25 app.delete('/api/hello/:id', (req, res) => {
26   const { id } = req.params;
27   console.log("Deleting something....");
28   res.status(200).json({ message: `This is a ${req.method} request on ${req.originalUrl}. Deleting something with id : ${id}` });
29 });
30
31 // Start the server
32 app.listen(PORT, () => {
33   console.log(`Server is running on http://localhost:${PORT}`);
34 });
```

(backend/src/server.js)

3. Pada file **package.json**, tambahkan script berikut untuk dapat memulai server Express JS kita. Package nodemon berfungsi agar server ter-restart secara otomatis jika ada perubahan. Tanpa Nodemon, kita harus restart server secara manual.

```
backend > {} package.json > ...
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "main": "index.js",
5    >Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "dev": "nodemon src/server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "description": "",
14   "dependencies": {
15     "cors": "^2.8.5",
16     "express": "^5.1.0",
17     "nodemon": "^3.1.10"
18   }
19 }
```

(backend/package.json)

4. Jalankan server Express JS dengan command di terminal :

npm run dev

CORS Handling

CORS (Cross-Origin Resource Sharing) adalah mekanisme pengaturan izin akses terhadap suatu resource. Secara default, **browser** akan memblokir request dari origin yang berbeda (beda **domain**, **port**, atau **protokol**). Mekanisme ini mencegah situs asing mengambil data dari server backend kita tanpa otorisasi.

Jika kita tidak mengatur header CORS, **browser** akan **memblokir respon** dari backend jika server tidak memberikan izin melalui header HTTP Access-Control-Allow-Origin.

Contoh perbedaan origin:

- Same-origin (tidak kena CORS):
Frontend → <https://vedianbunka.com>
Backend → <https://vedianbunka.com/api>
- Cross-origin (terkena CORS):
Frontend → <https://vedianbunka.com>
Backend → <https://api.vedianbunka.com>

Frontend → <http://localhost:5173>
Backend → <http://localhost:5001>

Oleh karena itu, kita perlu mengatur origin mana saja yang dapat mengakses server backend kita tanpa diblokir oleh browser.

Mengaktifkan CORS ke Semua Origin di Express JS

1. Pada terminal, jalankan command berikut untuk menginstall package cors:
npm install cors
2. Pada **server.js**, tambahkan kode berikut untuk memberi akses terhadap semua origin.

```

1  const express = require('express');
2  const cors = require('cors');
3
4  const app = express();
5  const PORT = 5001;
6
7  // Middleware to enable CORS
8  app.use(cors());
9

```

(backend/src/server.js)

Membatasi CORS ke Origin Tertentu

Ubah konfigurasi CORS pada **server.js** untuk menentukan origin mana saja yang diizinkan mengakses API.

```

app.use(
  cors({
    origin: [
      "http://localhost:5173", // React app running on localhost
      "https://vedianbunka.com" // Production URL
      // Add other allowed origins here if needed
    ],
  })
);

```

(backend/src/server.js)

Setup Frontend

1. Kembali ke folder **root (Week11_NIMtanpa0)** dan install react app dengan menggunakan **vite** dengan nama project/folder **frontend**.

cd ..

npm create vite@latest frontend

```

(base) vedianbunka@Vedians-MacBook-Air Week11_91424 % npm create vite@latest frontend
> npx
> create-vite frontend

Select a framework:
  o Vanilla
  o Vue
  o React
  o Preact
  o Lit
  o Svelte
  o Solid
  o Quik
  o Angular
  o Marko
  o Others

(base) vedianbunka@Vedians-MacBook-Air Week11_91424 % npm create vite@latest frontend
> npx
> create-vite frontend

Select a framework:
  React

Select a variant:
  o TypeScript
  o TypeScript + SWC
  o JavaScript
  o JavaScript + SWC
  o React Router v7
  o TanStack Router
  o RedwoodSDK
  o RSC

```

2. Masuk ke folder **frontend** dan install dependencies yang diperlukan **node_modules**, **react-router** (untuk routing), dan **axios** (untuk fetch API). Jalankan comman ini pada terminal:

```
cd frontend
npm install
npm install react-router axios
npm run dev
```

3. Pada file `src/main.jsx`, bungkus `<App />` dengan **BrowserRouter** dari package **react-router** yang telah kita install sebelumnya. Jangan lupa untuk mengimportnya terlebih dahulu.

```
frontend > src > main.jsx
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import "./index.css";
4  import App from "./App.jsx";
5  import { BrowserRouter } from "react-router";
6
7  createRoot(document.getElementById("root")).render(
8    <StrictMode>
9      <BrowserRouter>
10       <App />
11     </BrowserRouter>
12   </StrictMode>
13 );
```

(frontend/src/main.js)

4. Pada folder `src`, buatlah folder **pages**. Di dalamnya, buatlah file **GetMethod.jsx**, **PostMethod.jsx**, **PutMethod.jsx**, dan **DeleteMethod.jsx**. Definisikan routing pada `src/App.jsx`.

```
WEEK11_91424 frontend > src > App.jsx > ...
> backend
> frontend
  > node_modules
  > public
  > src
    > assets
    > pages
      DeleteMethod.jsx
      GetMethod.jsx
      PostMethod.jsx
      PutMethod.jsx
    # App.css
    App.jsx
    # index.css
    main.jsx

1  import { Routes, Route } from "react-router";
2  import GetMethod from "../pages/GetMethod.jsx";
3  import PostMethod from "../pages/PostMethod.jsx";
4  import PutMethod from "../pages/PutMethod.jsx";
5  import DeleteMethod from "../pages/DeleteMethod.jsx";
6
7  const App = () => {
8    return (
9      <Routes>
10        <Route path="/get" element={ <GetMethod /> } />
11        <Route path="/post" element={ <PostMethod /> } />
12        <Route path="/put" element={ <PutMethod /> } />
13        <Route path="/delete" element={ <DeleteMethod /> } />
14      </Routes>
15    );
16  };
17
18  export default App;
```

(frontend/src/App.jsx)

Fetching API From Frontend (React JS) to Backend (Express JS)

1. Pada folder `src`, buatlah folder **lib** dan di dalamnya buatlah file **axios.js**. Kita akan membuat axios instance yang berguna untuk melakukan fetch API dengan library axios. Dengan membuat axios instance, ketika melakukan fetch API, kita tidak perlu menulis URL lengkap, hanya perlu pathnya

saja. Selain itu, kita dapat mengganti URL dengan mudah, menyesuaikan terhadap environment (development atau production).

```
frontend > src > lib > JS axios.js > ...
1   import axios from 'axios';
2
3   const api = axios.create({
4     baseURL: 'http://localhost:5001'
5   });
6
7   export default api;
```

(frontend/src/lib/axios.js)

2. Pada **src/pages/GetMethod.jsx**, salin kode berikut untuk melakukan fetch API ke server Express JS kita dengan GET method.

```
frontend > src > pages > GetMethod.jsx > ...
1   import { useState, useEffect } from 'react';
2   import api from '../lib/axios';
3
4   const GetMethod = () => {
5     const [data, setData] = useState(null); // State untuk menyimpan data dari API
6
7     useEffect(() => {
8       const getMethod = async () => {
9         try {
10           // Menggunakan axios instance yang telah dibuat di ../lib/axios.js untuk melakukan GET request
11           // Tanpa axios instance, kita perlu menulis URL lengkap axios.get("http://localhost:5001/api/hello")
12           // Namun, dengan axios instance, kita cukup menggunakan api.get("/api/hello")
13           // Ini membuat kode lebih bersih dan mudah dikelola
14
15           // Note : WAJIB menggunakan await untuk menunggu hasil dari request dan async keyword sebelum arrow function
16           // Jika tidak menggunakan async, maka kita tidak bisa menggunakan await di dalam fungsi tersebut
17           // Jika tidak menggunakan await, maka res akan berisi Promise, bukan hasil dari request
18           const res = await api.get("/api/hello");
19
20           // Menyimpan data yang diterima ke dalam state
21           // Pada axios, response dari server dapat diakses melalui res.data
22           // Server Express JS kita mengembalikan JSON dengan format { message: "..."}
23           // Jadi kita ambil res.data.message untuk menampilkan pesan
24           setData(res.data.message);
25         } catch (error) {
26           console.error("Error fetching data:", error);
27         }
28       };
29       getMethod();
30     }, []);
31
32     return <div>{data ? data : "Loading..."}</div>;
33   }
34
35   export default GetMethod;
```

(frontend/src/pages/GetMethod.jsx)

3. Pada **src/pages/PostMethod.jsx**, salin kode berikut untuk melakukan fetch API ke server Express JS kita dengan POST method.

```

frontend > src > pages > PostMethod.jsx > ...
1  import { useState } from "react";
2  import api from "../lib/axios";
3
4  const PostMethod = () => {
5    const [name, setName] = useState("");
6    const [data, setData] = useState();
7
8    const handleSubmit = async (e) => {
9      e.preventDefault();
10     try {
11       // Menggunakan axios instance yang telah dibuat di ../lib/axios.js untuk melakukan POST request
12
13       // Note : WAJIB menggunakan await untuk menunggu hasil dari request dan async keyword sebelum arrow function
14       // Jika tidak menggunakan async, maka kita tidak bisa menggunakan await di dalam fungsi tersebut
15       // Jika tidak menggunakan await, maka res akan berisi Promise, bukan hasil dari request
16       const res = await api.post("/api/hello", { name });
17
18       // Menyimpan data yang diterima ke dalam state
19       // Pada axios, response dari server dapat diakses melalui res.data
20       // Server Express JS kita mengembalikan JSON dengan format { message: "..."}
21       // Jadi kita ambil res.data.message untuk menampilkan pesan
22       setData(res.data.message);
23     } catch (error) {
24       console.error("Error posting data:", error);
25     }
26   };
27
28   return (
29     <div>
30       <h1>Post Method</h1>
31       <p>{data} && "Response API : " + data</p>
32       <form>
33         <label htmlFor="name">Name:</label>
34         <input
35           type="text"
36           placeholder="Vedian Bunka"
37           onChange={e => setName(e.target.value)}
38         />
39         <br />
40         <button type="submit" onClick={handleSubmit}>
41           Submit
42         </button>
43       </form>
44     </div>
45   );
46 };
47
48 export default PostMethod;

```

(frontend/src/pages/PostMethod.jsx)

4. Pada **src/pages/PutMethod.jsx**, salin kode berikut untuk melakukan fetch API ke server Express JS kita dengan PUT method.

```

frontend > src > pages > PutMethod.jsx > ...
1  import { useState } from "react";
2  import api from "../lib/axios";
3
4  const PutMethod = () => {
5    const [name, setName] = useState("Vedian Bunka");
6    const [data, setData] = useState();
7
8    const handleSubmit = async (e) => {
9      e.preventDefault();
10     try {
11       // Menggunakan axios instance yang telah dibuat di ../lib/axios.js untuk melakukan PUT request
12
13       // Note : WAJIB menggunakan await untuk menunggu hasil dari request dan async keyword sebelum arrow function
14       // Jika tidak menggunakan async, maka kita tidak bisa menggunakan await di dalam fungsi tersebut
15       // Jika tidak menggunakan await, maka res akan berisi Promise, bukan hasil dari request
16       const res = await api.put("/api/hello", { name });
17
18       // Menyimpan respon server yang diterima ke dalam state
19       // Pada axios, response dari server dapat diakses melalui res.data
20       // Server Express JS kita mengembalikan JSON dengan format { message: "..." }
21       // Jadi kita ambil res.data.message untuk menampilkan pesan
22       setData(res.data.message);
23     } catch (error) {
24       console.error("Error posting data:", error);
25     }
26   };
27
28   return (
29     <div>
30       <h1>Put Method</h1>
31       <p>{data} && "Response API : " + data</p>
32       <form>
33         <label htmlFor="name">Name:</label>
34         <input
35           type="text"
36           value={name}
37           onChange={(e) => setName(e.target.value)}
38         />
39         <br />
40         <button type="submit" onClick={handleSubmit}>
41           Submit
42         </button>
43       </form>
44     </div>
45   );
46 };
47
48 export default PutMethod;

```

(frontend/src/pages/PutMethod.jsx)

5. Pada **src/pages/DeleteMethod.jsx**, salin kode berikut untuk melakukan fetch API ke server Express JS kita dengan DELETE method.


```

frontend > src > pages > DeleteMethod.jsx > ...
1  import { useState } from "react";
2  import api from "../lib/axios";
3
4  const DeleteMethod = () => {
5    const [id, setId] = useState("");
6    const [data, setData] = useState();
7
8    const handleSubmit = async (e) => {
9      e.preventDefault();
10     try {
11       // Menggunakan axios instance yang telah dibuat di ../lib/axios.js untuk melakukan DELETE request
12
13       // Note : WAJIB menggunakan await untuk menunggu hasil dari request dan async keyword sebelum arrow function
14       // Jika tidak menggunakan async, maka kita tidak bisa menggunakan await di dalam fungsi tersebut
15       // Jika tidak menggunakan await, maka res akan berisi Promise, bukan hasil dari request
16       const res = await api.delete("/api/hello/" + id);
17
18       // Menyimpan data yang diterima ke dalam state
19       // Pada axios, response dari server dapat diakses melalui res.data
20       // Server Express JS kita mengembalikan JSON dengan format { message: ".." }
21       // Jadi kita ambil res.data.message untuk menampilkan pesan
22       setData(res.data.message);
23     } catch (error) {
24       console.error("Error posting data:", error);
25     }
26   };
27
28   return (
29     <div>
30       <h1>Delete Method</h1>
31       <p>{data} && "Response API : " + data</p>
32       <form>
33         <label htmlFor="name">ID:</label>
34         <input
35           type="text"
36           placeholder="Masukkan ID untuk dihapus"
37           onChange={(e) => setId(e.target.value)}
38         />
39         <br />
40         <button type="submit" onClick={handleSubmit}>
41           Submit
42         </button>
43       </form>
44     </div>
45   );
46 }
47
48 export default DeleteMethod

```

(frontend/src/pages/PutMethod.jsx)

Tugas

1. Buatlah frontend dengan React JS dengan ketentuan sebagai berikut:
 - Menggunakan REST API dari week 10.
 - Terdapat CORS handling yang hanya meng-allow 1 origin saja : URL frontend.
 - Terdapat fitur CRUD (Create, Read, Update, Delete).
 - Terdapat fitur **search** yang didapatkan dengan **fetch API** ke backend. Pastikan terdapat endpoint search pada backend.
 - Tampilan interface yang menarik.
 - Kumpul dengan format penamaan "**Week11_NIMtanpa0_Tugas**". Hapus terlebih dahulu folder **node_modules** dari frontend dan backend.